📋 GUIDE COMPLET DE DÉPLOIEMENT EN PRODUCTION

🎯 Vue d'ensemble des problèmes rencontrés et solutions

Problème	Cause	Prévention	Priorité
Analyse d'anomalies échoue	`profil_client_id` vides	UPDATE SQL + Code corrigé	**HAUTE**
Connexion échoue	Email case-sensitive	Recherche case-insensitive	**HAUTE**
AutoMigrate échoue	Contrainte mal nommée	`DB_AUTO_MIGRATE=false`	**MOYENNE**
Meta-données manquantes	Table pas peuplée	INSERT SQL initialisation	**HAUTE**
Noms de table incorrects	Incohérence BD/Code	Vérification cohérence	**MOYENNE**

📦 PRÉ-DÉPLOIEMENT - Checklist

1. Variables d'environnement (OBLIGATOIRES)

```
# Backend
PARTNER_BANK_CODE=SN079                    # Code banque du partenaire
DB_AUTO_MIGRATE=false                      # IMPORTANT: Désactiver
AutoMigrate
KEY_JWT_PRIVATE_KEY_FILE=/app/keys/jwt_private.pem
KEY_JWT_ACCESS_TTL_MIN=60                  # TTL des tokens (minutes)
PORT=8080

# Logs
LOG_LEVEL=info
```

Vérification:

```
docker exec <container_backend> env | grep -E
"PARTNER_BANK_CODE|DB_AUTO_MIGRATE"
# Résultat attendu:
# PARTNER_BANK_CODE=SN079
# DB_AUTO_MIGRATE=false
```

🗄️ MIGRATIONS DE BASE DE DONNÉES (À exécuter AVANT de redéployer)

1. Corriger les profil_client_id vides

```sql
-- Remplir les profil_client_id vides pour les comptes existants
UPDATE comptes c
SET profil_client_id = p.id::text
FROM profils_clients p
WHERE (c.profil_client_id IS NULL OR c.profil_client_id = '')
AND (
    (c.type_client = 'P' AND LOWER(p.nom_profil) = 'particulier')
    OR (c.type_client = 'M' AND LOWER(p.nom_profil) = 'entreprise')
    OR (c.type_client = 'C' AND LOWER(p.nom_profil) = 'commerçant')
    OR (c.type_client = 'B' AND LOWER(p.nom_profil) = 'business')
);

-- Vérifier qu'aucun profil_client_id n'est vide
SELECT COUNT(*) as comptes_sans_profil
FROM comptes
WHERE profil_client_id IS NULL OR profil_client_id = '';
-- Résultat: 0
```

2. Renommer table alert_meta → alertes_meta

```sql
-- Vérifier le nom actuel
SELECT table_name FROM information_schema.tables
WHERE table_name IN ('alert_meta', 'alertes_meta');

-- Renommer si nécessaire
ALTER TABLE alert_meta RENAME TO alertes_meta;

-- Vérifier
SELECT COUNT(*) FROM alertes_meta;
-- Résultat: 7 lignes
```

3. Insérer méta-données d'alertes

```sql
-- Supprimer les anciennes si elles existent
TRUNCATE TABLE alertes_meta CASCADE;

-- Insérer les 7 méta-données essentielles
INSERT INTO alertes_meta (code, type, statut, message, niveau_criticite)
VALUES
('A001', 'Montant', 'En cours', 'Cumul sur 7 jours dépassant le seuil
autorisé.', 'Haute'),
('A002', 'frequence', 'En cours', 'Nombre de transactions sur 24h
dépassant le seuil autorisé.', 'Haute'),
('A003', 'Pays à risque', 'En cours', 'Transaction vers ou depuis un pays
à risque.', 'Haute'),
('A004A', 'Interdit', 'En cours', 'Compte interdit : transaction
refusée.', 'Haute'),
('A004B', 'Blacklist', 'En cours', 'Compte blacklisté : transaction
```

```
validée sous surveillance.', 'Haute'),
('A004C', 'PEP', 'En cours', 'Personne Politiquement Exposée : transaction
validée sous surveillance.', 'Moyenne'),
('A004D', 'Surveillés', 'En cours', 'Compte sous surveillance :
transaction validée avec suivi renforcé.', 'Moyenne');

-- Vérifier
SELECT COUNT(*) FROM alertes_meta;
-- Résultat: 7
```

4. Vérifier les profils clients

```
-- Les 4 profils doivent exister
SELECT nom_profil FROM profils_clients ORDER BY nom_profil;
-- Résultat:
-- Business
-- Commerçant
-- Entreprise
-- Particulier
```

5. Vérifier les règles et pays à risque

```
-- Au moins 1 règle "pays_risque" active
SELECT COUNT(*) FROM regles_controle
WHERE type_regle = 'pays_risque' AND active = true;

-- Au moins 1 pays à risque (ex: IR pour Iran)
SELECT COUNT(*) FROM pays_risque
WHERE active = true;
```

🔧 DÉPLOIEMENT - Étapes

Phase 1: Préparation (30 min avant)

```
# 1. Vérifier l'état de la BD
psql -h <host> -U <user> -d anomaly_db -c "SELECT count(*) FROM comptes;"
psql -h <host> -U <user> -d anomaly_db -c "SELECT count(*) FROM
alertes_meta;"

# 2. Exécuter les migrations SQL (voir section précédente)
# 3. Vérifier les résultats

# 4. Sauvegarder l'ancien binaire
cp /path/to/api /path/to/api.backup.$(date +%Y%m%d_%H%M%S)
```

Phase 2: Déploiement du nouveau code

```
# 1. Compiler le nouveau binaire (en local)
cd /Users/mac/Desktop/anomaly-detector/backend
go build -o api cmd/api/main.go

# 2. Copier sur serveur prod
scp api <user>@<prod_server>:/path/to/api

# 3. Ou mettre à jour l'image Docker
docker build -t anomaly-detector:latest .
docker push <registry>/anomaly-detector:latest
```

Phase 3: Redémarrage du service

```
# 1. Arrêter le conteneur
docker stop <container_id>

# 2. Redémarrer
docker start <container_id>

# 3. Vérifier qu'il démarre sans erreur
docker logs <container_id> --tail 50

# Attendu: Aucune erreur AutoMigrate
# Vérifier présence de: "Listening and serving HTTP on :8080"
```

✅ TESTS POST-DÉPLOIEMENT - Validation complète

1. Test de connexion

```
curl -X POST http://<prod_url>:8080/api/v1/auth/login \
  -H "Content-Type: application/json" \
  -d '{
    "email": "admin@gsie.sn",
    "password": "admin123"
  }'

# Résultat attendu: 200 OK avec access_token
```

2. Test d'analyse de montant

```
curl -X POST http://<prod_url>:8080/api/v1/anomalies/analyze \
  -H "Content-Type: application/json" \
```

```
  -H "Authorization: Bearer <token>" \
  -d '{
    "nom_payeur": "Test Montant",
    "numero_compte_payeur": "SN079000100123456789012 99",
    "type_client_payeur": "P",
    "type_compte_payeur": "Courant",
    "categorie_payeur": "Salarié",
    "pays_payeur": "Senegal",
    "ville_payeur": "Dakar",
    "Adresse_payeur": "Test",
    "montant": 500000.0,
    "portee": "National",
    "nom_paye": "Test Beneficiaire",
    "numero_compte_paye": "SN079000400998877665544 99",
    "type_client_paye": "B",
    "type_compte_paye": "Epargne",
    "categorie_paye": "Entreprise",
    "pays_paye": "SN",
    "ville_paye": "Dakar",
    "Adresse_paye": "Test"
  }'

# Résultat attendu:
# - status: "transaction_bloquee" ou "anomalie_detectee"
# - alertes avec code "A001" (Montant)
```

3. Test d'analyse de pays à risque

```
curl -X POST http://<prod_url>:8080/api/v1/anomalies/analyze \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer <token>" \
  -d '{
    "nom_payeur": "Test Pays Risque",
    "numero_compte_payeur": "SN079000100123456789012 98",
    "type_client_payeur": "P",
    "type_compte_payeur": "Courant",
    "categorie_payeur": "Salarié",
    "pays_payeur": "Senegal",
    "ville_payeur": "Dakar",
    "Adresse_payeur": "Test",
    "montant": 100000.0,
    "portee": "National",
    "nom_paye": "Test Iran",
    "numero_compte_paye": "SN079000400998877665544 98",
    "type_client_paye": "B",
    "type_compte_paye": "Epargne",
    "categorie_paye": "Entreprise",
    "pays_paye": "IR",
    "ville_paye": "Teheran",
    "Adresse_paye": "Iran"
  }'
```

```
# Résultat attendu:
# - status: "anomalie_detectee"
# - alertes avec code "A003" (Pays à risque)
# - paysRisque: "IRAN"
```

4. Vérifier les logs

```
docker logs <container_id> | grep -E "ERROR|WARN" | head -20

# Ne devrait PAS contenir:
# - AutoMigrate failed
# - meta-donnée d'alerte introuvable
# - constraint does not exist
```

📝 CORRECTIONS AU CODE (À inclure)

Ces fichiers doivent être modifiés et déployés:

1. **backend/internal/store/profiles.go** (ligne ~25)
   - Changement: `LOWER(nom_profil) = LOWER(?)`
2. **backend/internal/store/users.go** (ligne ~165)
   - Changement: `LOWER(email) = LOWER(?)`
3. **backend/internal/api/handlers/anomalies.go** (lignes ~55-88)
   - Ajout: Gestion des erreurs de `GetProfilClientIDByCode`
4. **backend/internal/models/models.go** (lignes ~13-22)
   - Ajout: Tags GORM pour `Utilisateur` (column mapping)

🚨 ROLLBACK - En cas de problème

```
# 1. Restaurer l'ancien binaire
cp /path/to/api.backup.<date> /path/to/api

# 2. Redémarrer le service
docker restart <container_id>

# 3. Vérifier
docker logs <container_id> --tail 50

# 4. Notifier l'équipe et analyser les logs
```

📊 DOCUMENT DE SUIVI POST-DÉPLOIEMENT

À remplir après chaque déploiement:

- ☐ Date/Heure déploiement: _____
- ☐ Binaire déployé: api-_____
- ☐ Migrations SQL exécutées: Oui / Non
- ☐ Tests de connexion: ✅ / ❌
- ☐ Tests d'analyse montant: ✅ / ❌
- ☐ Tests d'analyse pays risque: ✅ / ❌
- ☐ Logs sans erreur: ✅ / ❌
- ☐ Utilisateurs notifiés: Oui / Non
- ☐ Observations: _____

🔗 Fichiers de référence

- Migrations SQL: backend/INSERT_ALERTES_META.sql
- Renommage table: backend/RENOMMER_TABLE_ALERTES_META.txt
- Corrections profils: backend/PROD_DIAGNOSTIC_QUERIES.sql
- Build script: backend/build.sh
- Checklist: Ce fichier

📞 Support et Questions

En cas de problème lors du déploiement:

1. Consulter les logs du conteneur: `docker logs <container_id>`
2. Vérifier les variables d'environnement
3. Re-exécuter les migrations SQL
4. Contacter l'équipe dev